

计算机组成原理第二次实验报告

实验名称：定点乘法器优化

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验目的

1. 理解迭代乘法的实现算法的原理，掌握基本实现算法。
2. 学习其他算法，尝试对迭代乘法算法进行优化。

二、实验内容

1. 阅读实验箱相关文档，学习乘法器基础原理，将给出的代码配置为项目并验证。
2. 深入学习计算机中乘法器的原理，并依此设计优化，绘制逻辑结构图。在本次实验中，主要包含两种优化算法的实现：
 - 两位迭代乘法.
 - 华莱士树.
3. 根据设计的优化方案，使用 verilog 修改核心模块，编写相应代码并进行仿真验证，得到正确的波形图。
4. 生成位流文件，将其导入实验箱烧录，验证实验箱各项功能。
5. 对比分析算法的原理差异，总结提高算法效率的优化思路。

三、实验原理图

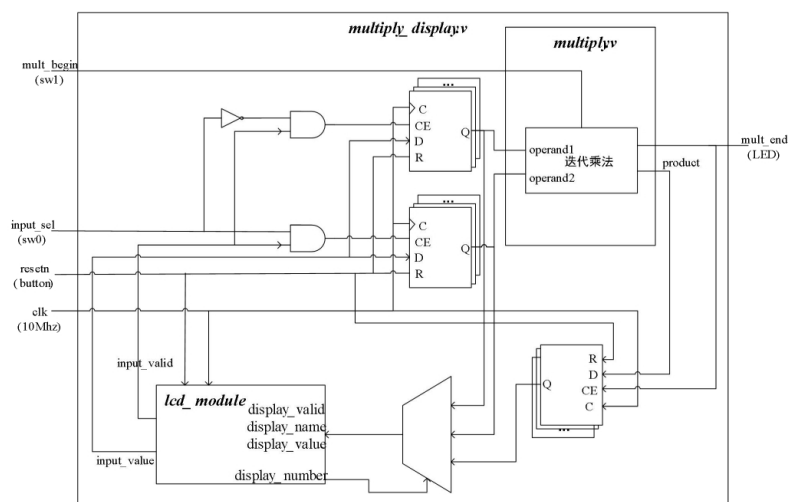


图 1: 实验原理图：模块总览

依照指导书的原理图，我们可以看出，原先的乘法器本质上是一位迭代的算法。在本实验中，外层显示与约束均已配置完毕，我们只需对 multiply.v 模块进行修改优化即可

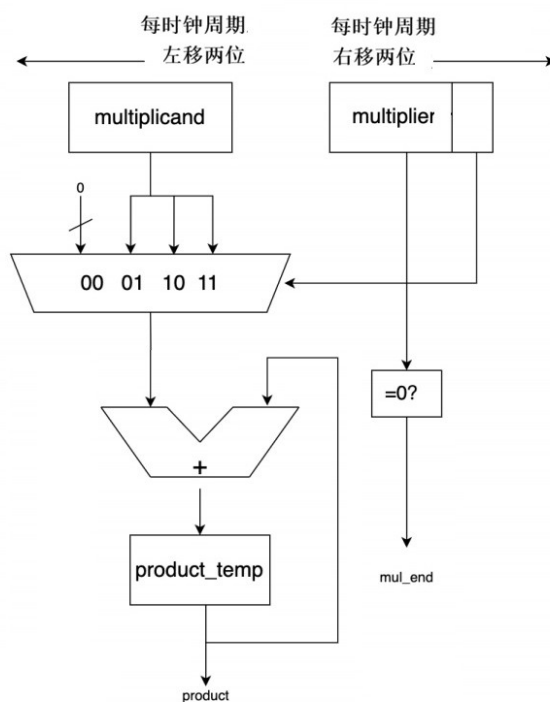


图 2: 乘法器模块：二位迭代优化

在一位迭代的基础上，把每个周期读取的位数从低一位改为低两位，并计算对应的部分积进行累加，从而使计算的总时钟周期数减少一半。

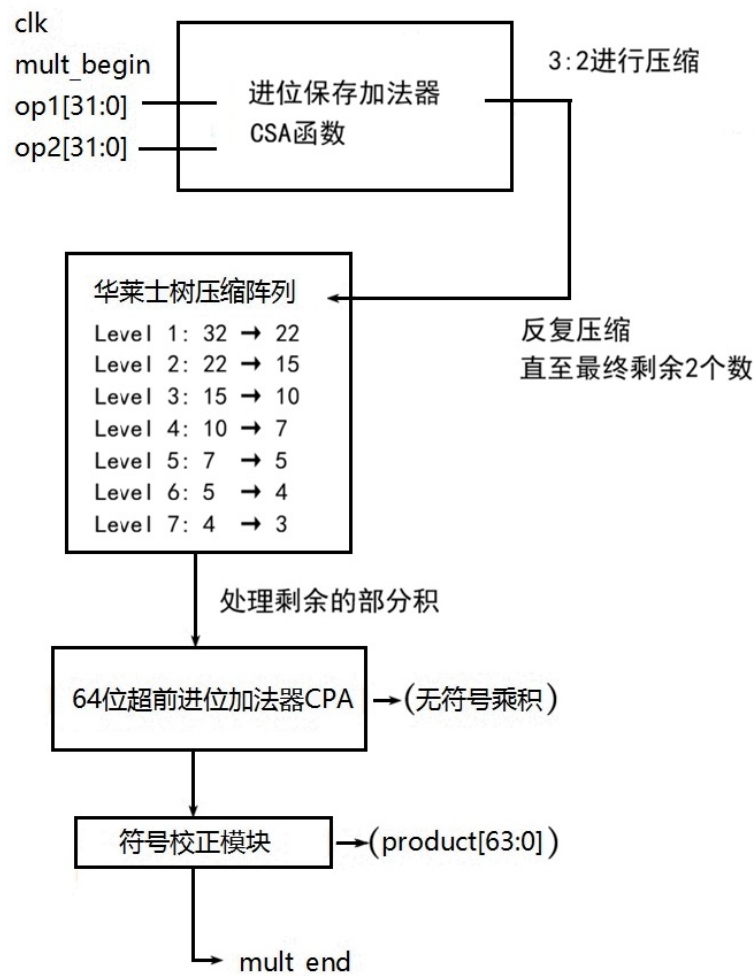


图 3: 乘法器模块：华莱士树优化

为了进一步提升乘法器的效率，采用华莱士树算法进行优化。作为树形加法器算法的代表，华莱士树算法主要以组合逻辑为主，通过大量的进位保存加法器并行地压缩部分积，从而大幅减少运算所需的时钟周期数。在实现思路，首先写一个进位保存加法器的函数用于后续压缩，然后将 32 个部分积反复经过华莱士树的压缩过程，逐步减少部分积的数量，在最后仅需计算剩余的两个数并处理符号，即可得到正确的结果。

由于华莱士树算法整体较为复杂，上图仅展示简略流程，详细实现将在后续的代码部分进一步分析讲解。

四、实验过程

1. 基础乘法器测试

使用 Vivado 创建项目文件，将老师提供的项目代码导入（为防止项目配置问题，此处使用文本级的复制粘贴进行导入），依据指导书完成设计文件、仿真文件、约束文件各部分，检查代码无误后，依次运行 Synthesis 与 Implementation，最后生成位流文件 Bitstream；代码项目生成完毕后，连接实验箱，将位流文件烧录至开发板上，依照设计进行功能验证。

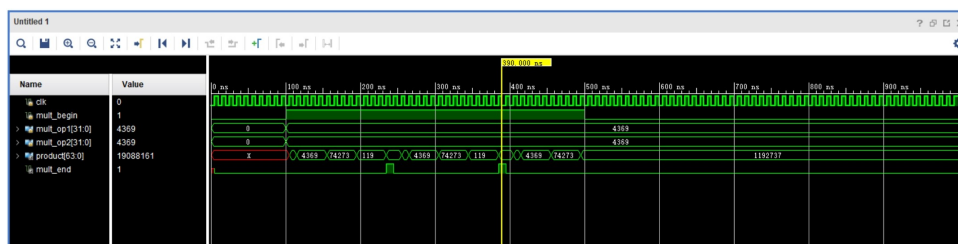


图 4: 仿真波形验证

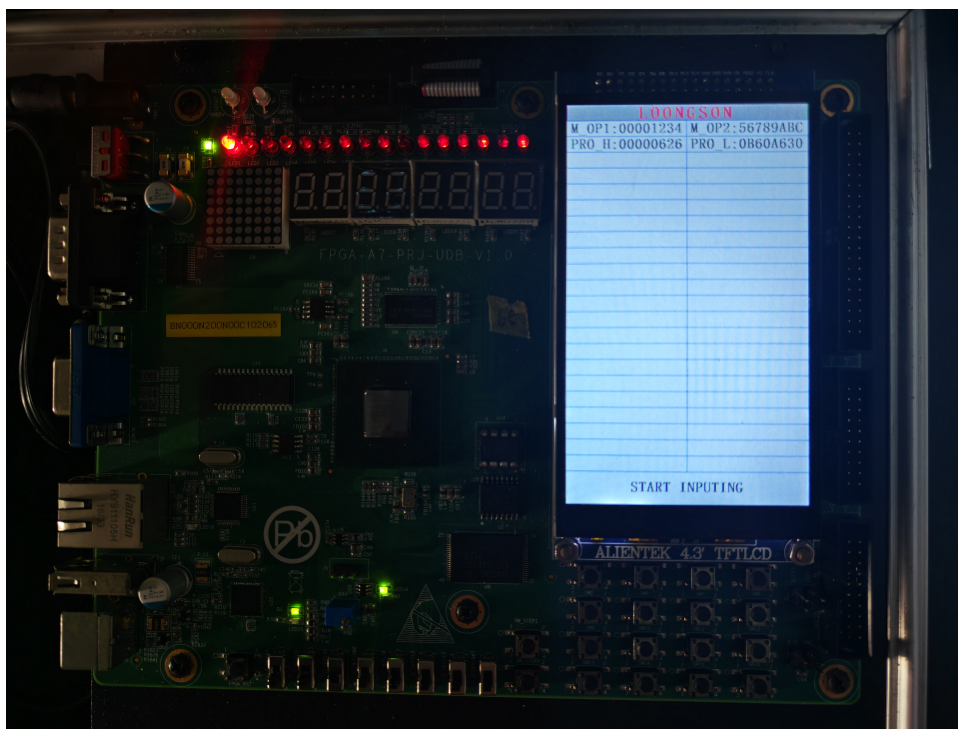


图 5: 实验箱验证

根据上图验证计算结果：

- $4369 \times 4369 = 19088161$ ，计算结果正确；
- $0x00001234 \times 0x56789ABC = 0x00000626|0B60A630$ ，计算结果正确，高位低位显示均正常。

经测验，乘法器各功能正常符合预期设计，LCD 屏也能正常显示，项目配置与代码逻辑均准确无误。接下来将以此为基础，对其进行优化改进，使其可以在更少的时钟周期数内完成计算。

2. 两位迭代乘法优化

完成了基础的一位迭代乘法器后，依据实验要求，采用两位迭代乘法对其进行优化。在这部分，仅需修改核心的乘法器模块 multiply.v 即可，其余代码无需改动。

i. 代码优化 (multiply.v)

```
1 // 被乘数改为每次左移2位
2 reg [63:0] multiplicand;
```

```

3      always @ (posedge clk)
4      begin
5          if (mult_valid)
6              begin // 被乘数每时钟左移2位
7                  multiplicand <= {multiplicand[61:0], 2'b00};
8              end
9          else if (mult_begin)
10             begin
11                 multiplicand <= {32'd0, op1_absolute};
12             end
13     end
14     // 乘数改为每次右移2位
15     reg [31:0] multiplier;
16     always @ (posedge clk)
17     begin
18         if (mult_valid)
19             begin // 乘数每时钟右移2位
20                 multiplier <= {2'b00, multiplier[31:2]};
21             end
22         else if (mult_begin)
23             begin
24                 multiplier <= op2_absolute;
25             end
26     end

```

首先，将乘数与被乘数在每时钟周期迭代时的移位数由一位改为两位。与基础的一位迭代乘法相比，在每次迭代中，所得的部分积变为了乘数的低两位乘以被乘数。为了容纳这样的多位数，在后面的部分积部分，还需改为采用两个部分积数进行处理。

```

1      // 同时生成两个部分积（对应乘数的第0、1位）
2      // 部分积1：对应乘数第0位，直接用被乘数
3      // 部分积2：对应乘数第1位，需要将乘数左移1位
4      wire [63:0] partial_product1;
5      wire [63:0] partial_product2;
6      assign partial_product1 = multiplier[0] ? multiplicand : 64'd0;
7      assign partial_product2 = multiplier[1] ? {multiplicand[62:0], 1'b0} : 64'd0;
8
9      // 累加器改为同时加两个部分积
10     reg [63:0] product_temp;
11     always @ (posedge clk)
12     begin
13         if (mult_valid)
14             begin
15                 product_temp <= product_temp + partial_product1 + partial_product2;
16             end
17         else if (mult_begin)
18             begin
19                 product_temp <= 64'd0; // 乘法开始，乘积清零
20             end
21     end

```

在这里，我们同时生成两个部分积分别对应乘数的低两位。需要注意的是，考虑到数位的计算，此处较高的一位需要进行 1 次左移操作才能符合对应的数值。最后，将累加器也同步改为同时加上两个部分积，即可实现完整的两位迭代乘法。

ii. 结果验证

代码修改完成后，先运行仿真文件进行初步验证，得到以下仿真波形。

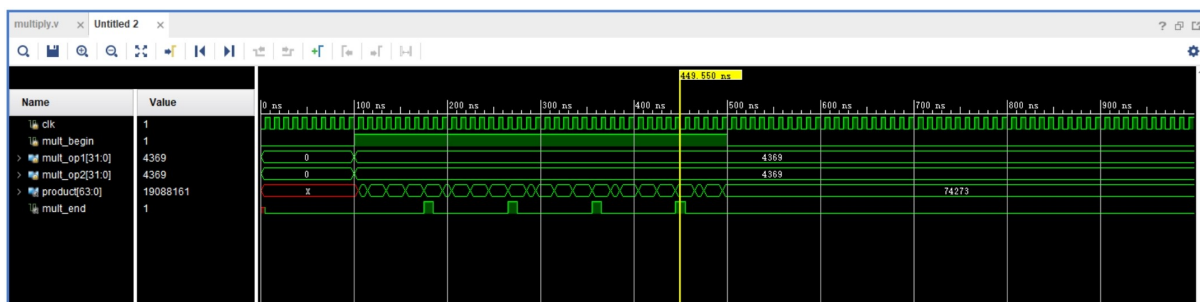


图 6: 仿真波形输出

运算数值与基础的一位迭代乘法器一致，计算结果正确；在计算所需的时钟周期数上，相同时间内可以完成 4 次乘法运算，优于刚才的一位迭代乘法器的 2 次。对时间刻度进一步详细对比，可知其速度提升为两倍，完美符合实验原理与设计预期。

仿真波形确认无误后，依照流程生成位流文件烧录至实验箱，在实验箱上进行功能验证。

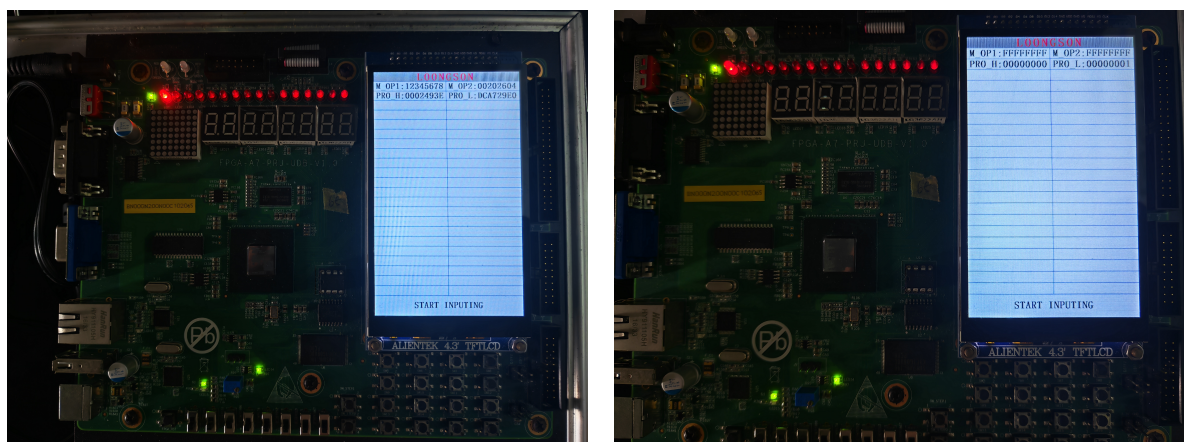


图 7: 计算运行结果

根据上图验证计算结果：

- 乘数：M_OP1 = 0x12345678（转换为十进制：305419896）
M_OP2 = 0x00202604（转换为十进制：2106884）
- 结果：PRO_H = 0x0002493E PRO_L = 0xDCA729E0
即：0x0002493EDCA729E0，转换为十进制：643484292164064
 $305419896 \times 2106884 = 643484292164064$ ，计算结果正确

- 乘数: $M_OP1 = 0xFFFFFFFF$ (转换为十进制: -1)
 $M_OP2 = 0xFFFFFFFF$ (转换为十进制: -1)
- 结果: $PRO_H = 0x00000000$ $PRO_L = 0x00000001$
 即: $0x0000000000000001$, 转换为十进制: 1
 $-1 \times -1 = 1$, 计算结果正确

乘法器计算得到的结果均正确, 完全符合实验预期, 验证了两位迭代算法的正确性与高效性。

3. 华莱士树优化

为了进一步优化乘法器的效率, 使其可以在更少的时钟周期内完成计算, 考虑使用华莱士树进行优化。

i. 代码优化 (multiply.v)

```

1  `timescale 1ns / 1ps
2
3  // 进位保留加法器模块
4  module csa (
5      input  [63:0] in1,
6      input  [63:0] in2,
7      input  [63:0] in3,
8      output [63:0] sum,
9      output [63:0] carry
10 );
11     assign sum    = in1 ^ in2 ^ in3;
12     assign carry = ((in1 & in2) | (in2 & in3) | (in1 & in3)) << 1;
13 endmodule
14
15
16 module multiply(                // 乘法器
17     input      clk,            // 时钟
18     input      mult_begin,     // 乘法开始信号
19     input  [31:0] mult_op1,     // 乘法源操作数1
20     input  [31:0] mult_op2,     // 乘法源操作数2
21     output [63:0] product,      // 乘积
22     output      mult_end       // 乘法结束信号
23 );
24
25     wire      op1_sign        = mult_op1[31];
26     wire      op2_sign        = mult_op2[31];
27     wire [31:0] op1_absolute = op1_sign ? (~mult_op1 + 1) : mult_op1;
28     wire [31:0] op2_absolute = op2_sign ? (~mult_op2 + 1) : mult_op2;
29
30     wire [63:0] pp [0:31];
31     genvar i;
32     generate
33         for (i = 0; i < 32; i = i + 1) begin : gen_pp

```

```

34         assign pp[i] = op2_absolute[i] ? ({32'd0, op1_absolute} << i) : 64'd0;
35     end
36 endgenerate
37
38
39 // 华莱士树压缩 (一共8层)
40
41 // Level 1: 32 -> 22
42 wire [63:0] l1_out [0:21];
43 generate
44     for(i = 0; i < 10; i = i + 1) begin : l1_csa
45         csa u_csa (pp[i*3], pp[i*3+1], pp[i*3+2], l1_out[i*2], l1_out[i*2+1]);
46     end
47 endgenerate
48 assign l1_out[20] = pp[30];
49 assign l1_out[21] = pp[31];
50
51 // Level 2: 22 -> 15
52 wire [63:0] l2_out [0:14];
53 generate
54     for(i = 0; i < 7; i = i + 1) begin : l2_csa
55         csa u_csa (l1_out[i*3], l1_out[i*3+1], l1_out[i*3+2], l2_out[i*2],
56                 l2_out[i*2+1]);
57     end
58 endgenerate
59 assign l2_out[14] = l1_out[21];
60
61 // Level 3: 15 -> 10
62 wire [63:0] l3_out [0:9];
63 generate
64     for(i = 0; i < 5; i = i + 1) begin : l3_csa
65         csa u_csa (l2_out[i*3], l2_out[i*3+1], l2_out[i*3+2], l3_out[i*2],
66                 l3_out[i*2+1]);
67     end
68 endgenerate
69
70 // Level 4: 10 -> 7
71 wire [63:0] l4_out [0:6];
72 generate
73     for(i = 0; i < 3; i = i + 1) begin : l4_csa
74         csa u_csa (l3_out[i*3], l3_out[i*3+1], l3_out[i*3+2], l4_out[i*2],
75                 l4_out[i*2+1]);
76     end
77 endgenerate
78 assign l4_out[6] = l3_out[9];
79
80 // Level 5: 7 -> 5
81 wire [63:0] l5_out [0:4];
82 generate
83     for(i = 0; i < 2; i = i + 1) begin : l5_csa

```

```

81         csa u_csa (14_out[i*3], 14_out[i*3+1], 14_out[i*3+2], 15_out[i*2],
82                 15_out[i*2+1]);
83     end
84 endgenerate
85
86 // Level 6: 5 -> 4
87 wire [63:0] 16_out [0:3];
88 csa 16_csa_inst (15_out[0], 15_out[1], 15_out[2], 16_out[0], 16_out[1]);
89 assign 16_out[2] = 15_out[3];
90 assign 16_out[3] = 15_out[4];
91
92 // Level 7: 4 -> 3
93 wire [63:0] 17_out [0:2];
94 csa 17_csa_inst (16_out[0], 16_out[1], 16_out[2], 17_out[0], 17_out[1]);
95 assign 17_out[2] = 16_out[3];
96
97 // Level 8: 3 -> 2
98 wire [63:0] 18_out [0:1];
99 csa 18_csa_inst (17_out[0], 17_out[1], 17_out[2], 18_out[0], 18_out[1]);
100
101 // 最后加法器处理剩余两个数
102 wire [63:0] tree_result;
103 assign tree_result = 18_out[0] + 18_out[1];
104
105
106 // 状态控制与结果输出
107 reg mult_valid;
108 assign mult_end = mult_valid;
109
110 always @(posedge clk) begin
111     if (!mult_begin || mult_end) begin
112         mult_valid <= 1'b0;
113     end else begin
114         mult_valid <= 1'b1;
115     end
116 end
117
118 // 锁存组合逻辑生成的树结果和符号位
119 reg [63:0] product_temp;
120 reg        product_sign;
121
122 always @(posedge clk) begin
123     if (mult_begin && !mult_valid) begin
124         product_temp <= tree_result;
125         product_sign <= op1_sign ^ op2_sign;
126     end
127 end
128
129 // 处理符号

```

```

130     assign product = product_sign ? (~product_temp + 1) : product_temp;
131
132 endmodule

```

代码实现思路主要依靠组合逻辑的进位保留加法器不断压缩，极大提升运算效率。具体实现方面，首先写一个进位保留加法器（CSA）的函数，然后不断对 32 个部分积进行压缩。在每一层的压缩过程中，每三个都会经过 CSA 的处理压缩为两个，剩余不足三个的则是直通进入下一层；如此反复处理，经过八层，处理后只剩下两个部分积，使用加法器直接相加得到无符号结果。最后处理一下输出与符号，即可得到最终结果。

ii. 结果验证

代码修改完成后，先运行仿真文件进行初步验证，得到以下仿真波形。

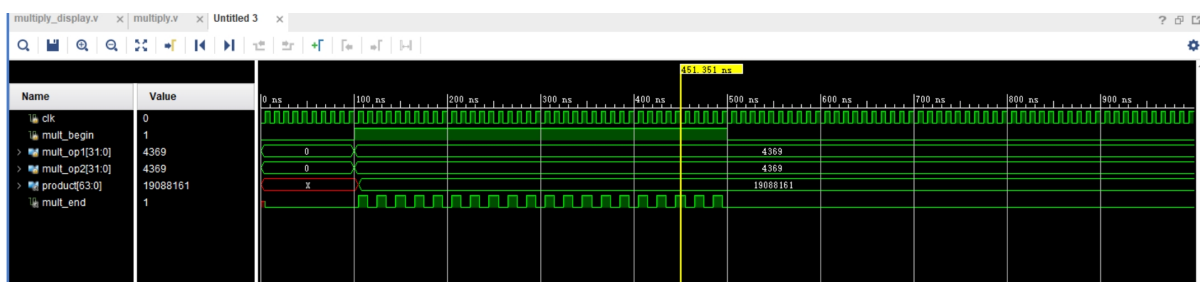


图 8: 仿真波形输出

运算数值与上述两种迭代乘法器一致，计算结果正确；在所耗时钟周期数上，其效率远远优于迭代乘法器，在验证了算法正确性的同时，还体现出组合逻辑在运算性能上的极大优势，完美符合实验原理与设计预期。

华莱士树的设计思路决定了其作为算法极高的性能优势，虽然在代码编写过程中，涉及的组合逻辑相对于迭代乘法器复杂许多——这意味着在实际工程中，需要复杂的布线与极高的结构设计成本——但是，其带来的性能提升还是相当可观的，像这样大限度缩短乘法器延时的算法，尤其适用于微电子与数字信号处理等高性能计算场景，同时也是许多优化算法的基础。

仿真波形确认无误后，依照流程生成位流文件烧录至实验箱，在实验箱上进行功能验证。

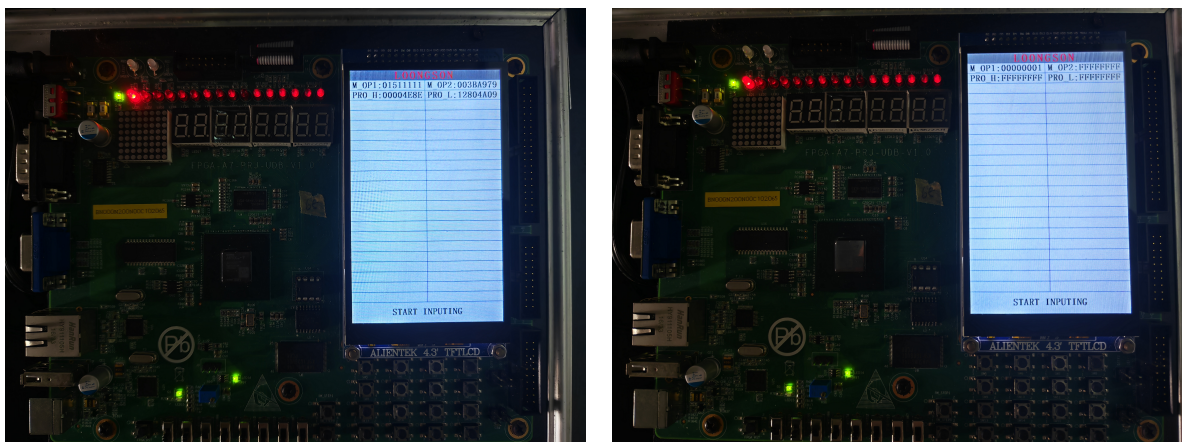


图 9: 计算运行结果

根据上图验证计算结果：

- 乘数: $M_OP1 = 0x01511111$ (转换为十进制: 22090001)
 $M_OP2 = 0x00202604$ (转换为十进制: 3910009)
- 结果: $PRO_H = 0x00004E8E$ $PRO_L = 0x12804A09$
 即: $0x00004E8E12804A09$, 转换为十进制: 86372102720009
 $22090001 \times 3910009 = 86372102720009$, 计算结果正确

- 乘数: $M_OP1 = 0x00000001$ (转换为十进制: 1)
 $M_OP2 = 0xFFFFFFFF$ (转换为十进制: -1)
- 结果: $PRO_H = 0xFFFFFFFF$ $PRO_L = 0xFFFFFFFF$
 即: $0xFFFFFFFFFFFFFFFF$, 转换为十进制: -1
 $-1 \times 1 = -1$, 计算结果正确

乘法器计算得到的结果均正确, 完全符合实验预期。

五、总结感想

本次实验中, 首先通过提供的源码实现了一位迭代乘法器的实现, 在此基础上进行优化, 分别使用两位迭代乘法和华莱士树两种算法实现了性能的提升。依照实验要求, 准确完成了本次实验。

通过这次实验, 我了解了乘法器的运算原理和代码实现方法。同时, 也在实践中对于算法的优化与性能提升的思路有了进一步的深入理解。迭代算法和树形算法分别从对时序电路流程优化与引入组合逻辑这两个角度切入, 使用不同的思路实现了性能的提升, 让我对于硬件的优化有了更多全新的认知; 此外, 在对新算法进行学习时的调研环节, 也让我开拓了工程层面的视野, 收获颇多。